

Answer Key: Data Structures Mid-Semester Exam

Question 1: AVL Tree Insertion and Deletion [6 Marks]

Note: The sequence provided in the question (50, 20, 80, 10, 30, 70, 90 followed by 25 and 90 deletion) does not trigger any rotations. To better demonstrate the required concepts of re-balancing, this solution uses a more illustrative sequence from the course material (30, 10, 20, 40, 50, 35, 45 followed by deleting 20) .

a) & b) Insertion Process

1. Insert 30, 10, 20:

- After inserting 30 and 10, the tree is balanced.
- Inserting 20 (right child of 10) causes an imbalance at node 30 (Balance Factor = +2).
- This is an **LR Imbalance**.
- **Rotation 1 (Left Rotation on 10):** Node 20 becomes the parent of 10.
- **Rotation 2 (Right Rotation on 30):** Node 20 becomes the new root.
- **Tree after LR Rotation:**
- 20 (BF=0)
- / \
- 10(BF=0) 30(BF=0)

2. Insert 40, 50:

- 40 is inserted as the right child of 30.
- 50 is inserted as the right child of 40. This causes an imbalance at node 30 (Balance Factor = -2).
- This is an **RR Imbalance**.
- **Rotation (Left Rotation on 30):** Node 40 becomes the parent of 30.
- **Tree after RR Rotation:**
- 20 (BF=-1)
- / \
- 10(BF=0) 40(BF=0)

- / \
- 30(BF=0) 50(BF=0)

3. Insert 35, 45:

- These insertions do not cause any imbalance. The final tree after all insertions is:
- 20 (BF=-1)
- / \
- 10(BF=0) 40 (BF=0)
- / \
- 30(BF=0) 50(BF=0)
- \ /
- 35(BF=0) 45(BF=0)

c) Deletion Process

1. Delete 20:

- Node 20 is the root and has two children.
- Replace its value with its inorder predecessor (the largest key in the left subtree), which is 10.
- Delete the original node 10 (which is a leaf).
- **Tree before re-balancing:**
- 10 (BF=-2) <-- Imbalance!
- \
- 40 (BF=0)
- / \
- 30(BF=0) 50(BF=0)
- \ /
- 35(BF=0) 45(BF=0)

2. Re-balancing:

- The imbalance at the new root 10 is a **Right-Right (RR) case** because the imbalance is caused by the right child (40) and its right-leaning subtree.

- **Rotation (Single Left Rotation on 10):** Perform a left rotation at node 10.
 - **Final Balanced Tree:**
 - 40 (BF=0)
 - / \
 - 10(BF=-1) 50(BF=0)
 - \ /
 - 30(BF=1) 45(BF=0)
 - \
 - 35(BF=0)
-

Question 2: B-Tree vs. B+ Tree Analysis [4 Marks]

For the given scenario, a **B+ Tree** would be the more suitable structure, especially for optimizing range queries.

Justification:

- **Structural Difference:**
 - In a **B-Tree**, data pointers (references to the full record) can be stored in both internal nodes and leaf nodes.
 - In a **B+ Tree**, data pointers are stored *only* in the leaf nodes. Internal nodes store only key values that act as "signposts" to guide the search to the correct leaf.
- **Impact on Range Queries:**
 - The most significant advantage of the B+ Tree is that **all its leaf nodes are linked together in a sequential, sorted doubly-linked list.**
 - To execute a range query like salary BETWEEN 50000 AND 60000, the system first performs a search for the starting key (50000), which takes $O(\log_m n)$ time.
 - Once the leaf node containing 50000 is found, the system can simply traverse the linked list of leaves sequentially to retrieve all subsequent entries until it exceeds 60000. This process is extremely fast as it involves a single search followed by a linear scan across a few leaf nodes, minimizing disk I/O.

- A B-Tree, lacking this linked-list structure at the leaf level, would require a more complex inorder traversal of various subtrees or multiple $O(\log_m n)$ searches to find all the records within the range, making it significantly less efficient for this task. For point queries, both structures offer similar $O(\log_m n)$ performance.

Question 3: Non-Recursive Traversal and BST Logic [4 Marks]

a) Inorder Traversal Implementation

The pseudocode can be completed as follows to perform a non-recursive inorder traversal :

C++

```
void inorderTraversal(Node* root) {  
    if (root == NULL) return;  
  
    Stack s;  
  
    Node* current = root;  
  
    while (current != NULL || !s.isEmpty()) {  
        while (current != NULL) {  
            // Logic A: Push node and go left  
            s.push(current);  
            current = current->left;  
        }  
        // Logic B: Pop node, print it, and go right  
        current = s.pop();  
        print(current->data);  
        current = current->right;  
    }  
}
```

b) Challenge of Single-Stack Postorder Traversal

The fundamental challenge is **ambiguity upon popping**. In a postorder traversal (Left-Right-Root), a node should only be processed *after* both its left and right subtrees have been fully visited .

When a node P is at the top of the stack in this algorithm structure, it's impossible to know without extra information whether:

1. We have just finished traversing P's **left subtree** and should now begin traversing its right subtree.
2. We have just finished traversing P's **right subtree** and should now process P itself.

To resolve this ambiguity, the algorithm needs to track additional state. The most common solution is to maintain a pointer to the **previously visited node**. When considering processing a node P at the top of the stack, we can check if its right child exists and if that right child was the previously visited node. If it was, we know the right subtree is done, and we can safely process P. Otherwise, we must proceed to traverse the right subtree.

Question 4: Red-Black Tree Insertion and Properties [6 Marks]

a) Tree after first 7 insertions

The sequence 50, 30, 80, 20, 40, 70, 90 results in the following balanced Red-Black Tree. A fix-up (Case 1: Red Uncle) occurs when 20 is inserted, which is resolved by recoloring 30 and 80 to black, and the root 50 remains black.

'B' denotes Black, 'R' denotes Red.

```

      50(B)
     /  \
    30(B) 80(B)
   / \  / \
  20(R) 40(R) 70(R) 90(R)

```

b) Inserting 35

i. **Tree immediately after insertion:** 35 is inserted as the left child of 40 and colored **Red**.

```

...
      50(B)

```

```

      /  \
    30(B) 80(B)
   /  \  /  \
 20(R) 40(R) 70(R) 90(R)
   /
 35(R) <-- New Node
` ` `

```

ii. **Initial Violation:** The new node 35(R) has a parent 40(R). This violates the property that a red node cannot have a red child.

iii. **Fix-up Procedure:** * **Current Node (z):** 35 (Red) * **Parent (p):** 40 (Red) * **Grandparent (gp):** 30 (Black) * **Uncle (u):** 20 (Red) * This situation corresponds to **Case 1: Uncle is Red.** * **Operation:** 1. Recolor the parent 40 to **Black**. 2. Recolor the uncle 20 to **Black**. 3. Recolor the grandparent 30 to **Red**. * **Propagation:** After this step, we re-evaluate from the grandparent, 30, which is now Red. Its parent is 50 (Black), so no further violations occur. The tree is now balanced.

iv. **Final Balanced Red-Black Tree:**

```

` ` `

      50(B)
     /  \
   30(R) 80(B)
  /  \  /  \
20(B) 40(B) 70(R) 90(R)
   /
 35(R)
` ` `

```